

Your forecast has a number. Does it have a decision?

A time-series forecasting cheat sheet
for the AI era.

- Start with the decision
- Choosing a horizon
- Baselines first
- Trend & seasonality
- Business calendars
- External drivers
- Prediction intervals
- Backtests & monitoring

Sagar Rathkanthiwar

Data & AI Professional

10 slides · save for later →

A forecast with no decision is just a chart

Nobody needs a number. They need to choose something. Before you model anything, name the decision the forecast is supposed to improve — and who makes it.

HOW MUCH INVENTORY DO WE ORDER?

Weekly units per SKU, per warehouse

Decided every Monday · owner: supply planning

HOW MANY STAFF DO WE SCHEDULE?

Hourly arrivals per store

Decided 2 weeks ahead · owner: ops manager

DO WE ADJUST THE BUDGET?

Quarterly revenue by region

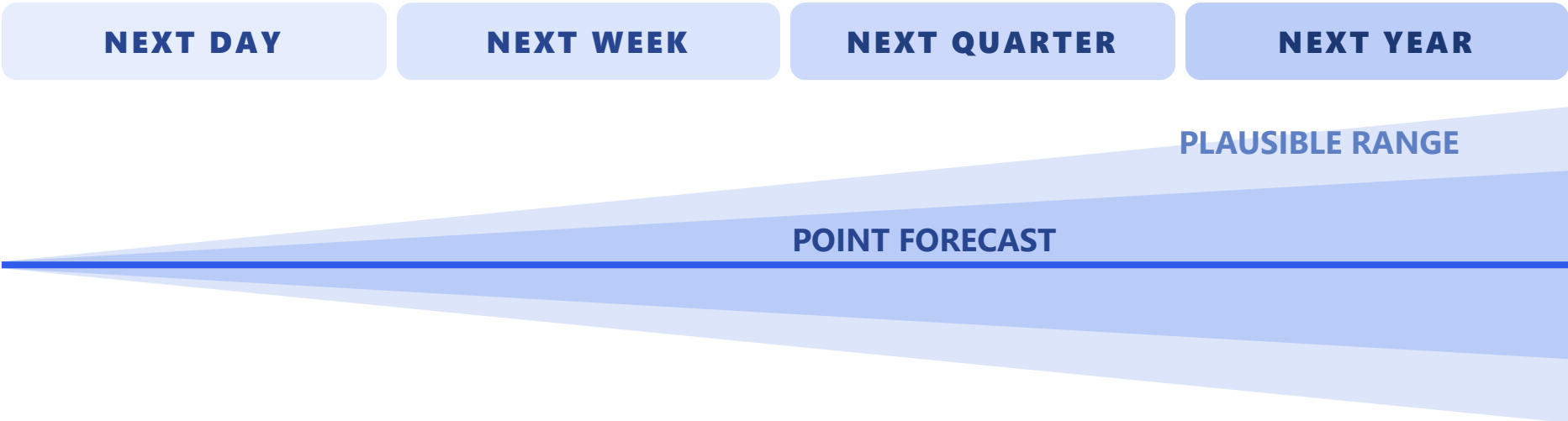
Decided at quarter close · owner: finance

THE FOUR QUESTIONS THAT DEFINE THE MODEL	
What is decided?	order quantity, headcount, budget, capacity
How often?	sets your forecast frequency — hourly, daily, weekly
How far ahead?	sets your horizon — and the lead time you must cover
What does being wrong cost?	over-forecast vs under-forecast are rarely equal

That last one reshapes everything. **A stockout may cost five times what excess inventory costs** — so the “best” forecast is not the most accurate one, it is the one that minimises the cost of being wrong.

How far ahead changes *everything*

Horizon is not a parameter you tune at the end. It decides your data, your model, and how much confidence you are entitled to.



uncertainty widens with every step you forecast ahead →

Next day

Recent momentum, hour-of-day patterns. Tight intervals. Drives dispatch, staffing, on-call.

Next week

Weekly seasonality and the promo calendar dominate. Drives ordering and rostering.

Next quarter

Trend, holidays, pricing. Intervals matter more than the point. Drives budgets and capacity.

Next year

Scenarios, not predictions. Range planning — low, base, high. Drives hiring and contracts.

Match the horizon to the lead time of the decision. If it takes six weeks to receive stock, a one-week forecast is accurate and useless — you needed the six-week number, intervals and all.

Beat the boring model, or don't ship the clever one

A baseline is a forecast so simple it needs no training. **If the clever model cannot beat it, it has earned nothing.**

Naive

tomorrow = today
 $\text{forecast}(t+1) = y(t)$

Seasonal naive

same day last week / same month last year
 $\text{forecast}(t+1) = y(t+1-s)$

Moving average

mean of the last k periods
 $\text{forecast}(t+1) = \text{mean}(\text{last } k \text{ values})$

```
# Seasonal naive: the forecast for a day is the same weekday one
# season ago.  season = 7 daily, 12 monthly, 24 hourly.
SEASON = 7

def seasonal_naive(history, horizon, season=SEASON):
    return [history[-season + (i % season)] for i in range(horizon)]

def backtest(y, horizon=7, season=SEASON):
    errors, start = [], 4 * season      # keep history before scoring
    for t in range(start, len(y) - horizon + 1, horizon):
        forecast = seasonal_naive(y[:t], horizon, season) # past only
        actual    = y[t:t + horizon]                     # never seen
        errors += [abs(f - a) for f, a in zip(forecast, actual)]
    return sum(errors) / len(errors)    # MAE, in units of the series

print("baseline MAE:", round(backtest(daily_units), 1))
```

Report your model **against** this number, not on its own: "MAE 42 vs a seasonal-naive 61 — a 31% improvement." **Model choice depends on your data and decision** — but the baseline comes first, every time.

Four things are happening in every series

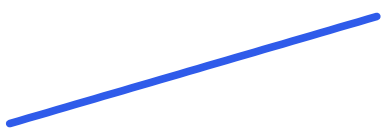
What you see is a sum. Learning to separate the parts tells you which model — and which mistakes — to expect.

WHAT YOU ACTUALLY OBSERVE — MONTHLY ACTIVE USERS



= trend + seasonality + cycle + noise

TREND



The long slope. Growth, decay, a step change after a launch.

SEASONALITY



Fixed period. Hour of day, day of week, month of year.

CYCLE



Variable length. Economic or product cycles — no fixed calendar.

NOISE



Irreducible. If you fit this, you have overfit.

Seasonality repeats on a calendar; a cycle does not. Confusing the two is the classic error — it makes a model extrapolate a boom as if it were a quarterly pattern.

The business calendar beats the algorithm

Most large forecast misses are not modelling failures. They are **a date the model did not know was special**.



- Weekend
- Payday
- Promotion
- Holiday

One month, four overlapping calendars. A model given only **the date** sees none of them.

Weekends & weekdays

Different demand shape entirely — and a month with five Saturdays is not like one with four.

Payday effects

The 1st, the 15th, month end. Retail and payments spike on a schedule the model cannot infer.

Promotions & launches

A self-inflicted spike. Without a promo flag, the model learns it as random noise.

Holidays & school terms

Moving dates — Easter, Diwali, Ramadan — plus the days *around* them.

Ask the people who live with the series: **“what happened on the weird days?”** That conversation adds more accuracy than switching algorithms — and it is knowledge no model can read out of the numbers.

A driver is only valid if you'll have it in time

Price, marketing spend, weather, rates — covariates can help a lot. But there is one test every one of them must pass first.

THE ONLY TEST THAT MATTERS

At the moment I produce the forecast, **will I already know this value for every future period I am forecasting?**

PASSES — KNOWN IN ADVANCE

Planned price & promo calendar
you set it yourself

Committed marketing spend
budget already signed off

Holidays, paydays, school terms
known for years ahead

Store openings, contracted volume
on the plan

FAILS — UNKNOWN AT FORECAST TIME

Actual weather next month
needs its own forecast — with its own error

Competitor pricing
you learn it afterwards

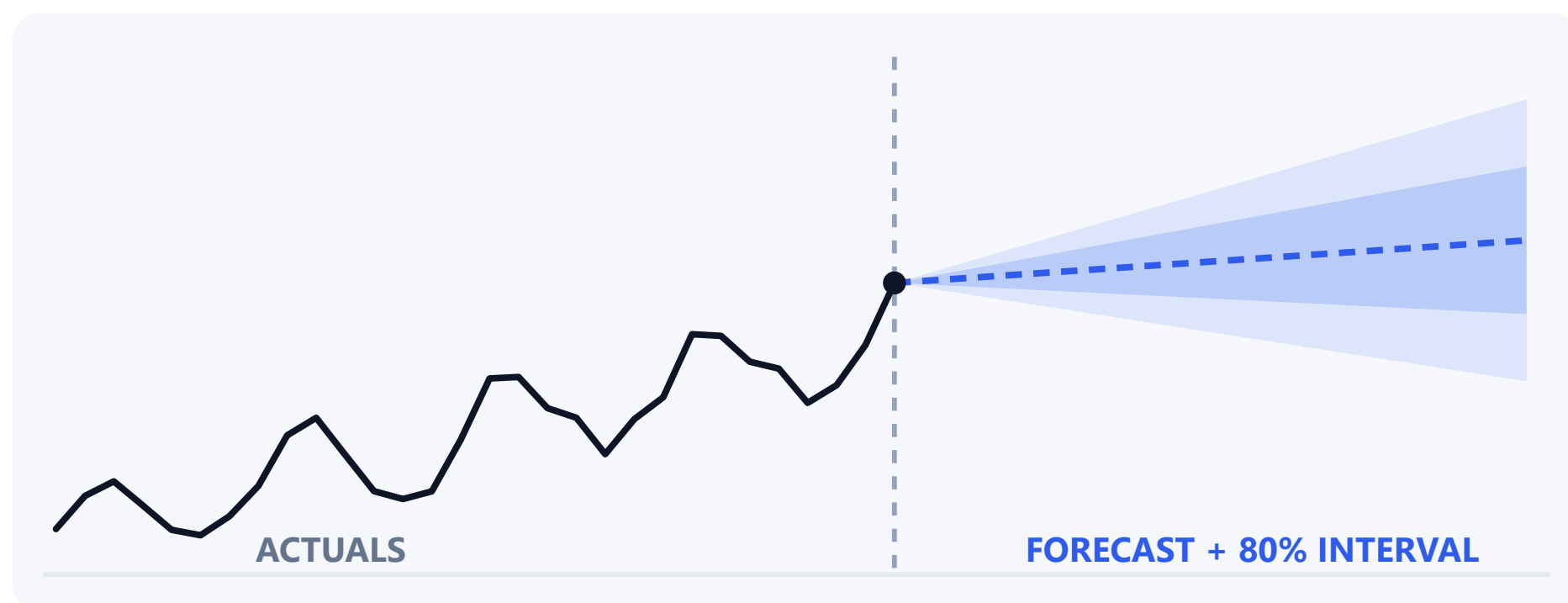
Same-period web traffic
arrives at the same time as the target

Economic indicators, unlagged
published weeks late — lag them or drop them

A failing driver is not always fatal — **lag it** (last month's indicator, yesterday's weather) or feed it a forecast and accept the compounded uncertainty. What you cannot do is train on the actual value and pretend you will have it.

“1,000 units” hides the number you needed

A point forecast is the middle of a distribution presented as a fact. **The width is the decision-relevant part.**

**POINT FORECAST****1,000**

units next week

80% INTERVAL**850 – 1,180**

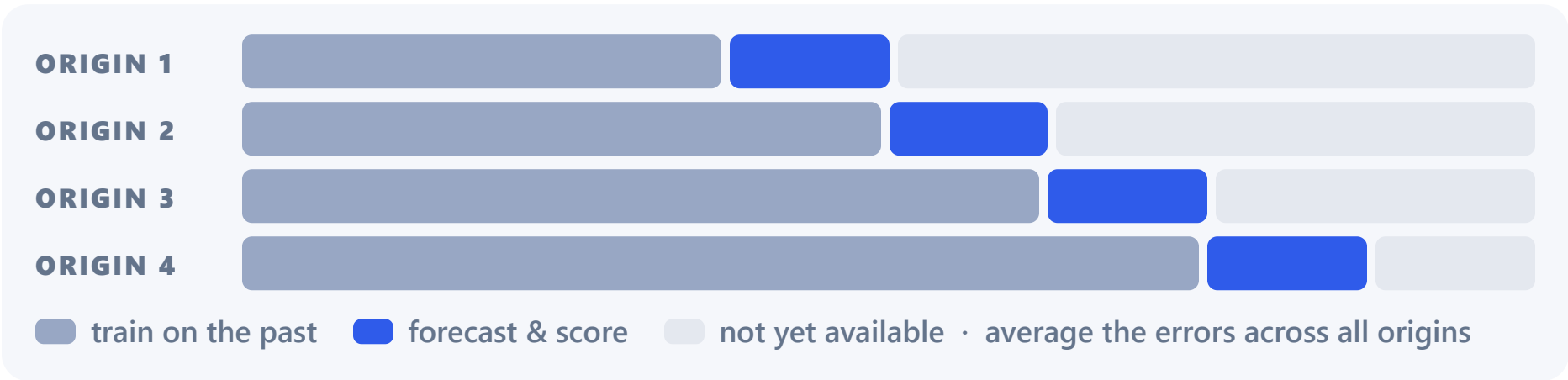
the planning range

WHAT YOU ORDER**1,180**if a stockout costs more than
excess

Read it out loud: **“we expect about 1,000, and in 8 weeks out of 10 it lands between 850 and 1,180.”** That sentence is what lets someone size a safety buffer. “1,000” alone quietly invites a plan with no slack in it.

Backtest forward. Then keep watching.

One train/test split tests one lucky week. **A rolling backtest re-forecasts at many origins** — the way production actually runs.



MAE

Average miss *in units*. “We are off by 42 units a day.” Easy to explain, easy to cost.

MAPE

Average miss *in percent*. Comparable across products — but breaks near zero and punishes under-forecasts less.

vs BASELINE

The only number that says whether the model was worth building. Always report it.

AFTER DEPLOYMENT — THE PART EVERYONE SKIPS

Chart **actual vs forecast** every cycle · watch for **bias** (missing the same direction repeatedly — a pattern you have not modelled) · alert when error drifts past a threshold · **retrain when the world changes**, not on a calendar reflex.

The goal is not to predict the future.

It is better decisions under uncertainty.

- **Name the decision first.** Frequency, horizon and the cost of being wrong all fall out of it.
- **Baseline before model.** Seasonal naive is the bar; report every result against it.
- **Separate trend, seasonality, cycle and noise** — and give the model the business calendar.
- **Only use drivers you will know in advance**, or lag them honestly.
- **Ship the range, not the point.** The interval is what makes a plan robust.
- **Backtest forward, then monitor.** A forecast is a system you operate, not a file you deliver.

AI can draft the model in seconds. It cannot tell you what decision this serves, whether a driver will exist at forecast time, or what a stockout costs you. **Defining the decision, validating the assumptions and owning the consequences stays human work.**

**What decision would a better forecast improve on your team?
Tell me below — that answer is the real model spec.**

Sagar Rathkanthiwar · Data & AI Professional · follow for more field guides

Note: the Python example is a baseline, not a recommendation. Model choice — naive, ETS, ARIMA, gradient boosting, deep learning — always depends on your data, horizon and decision context. Numbers shown are illustrative.